🐳 DOCKER

Sıfırdan İleri Seviyeye

3 Günlük Uygulamalı Eğitim Programı

Hedef Kitle: Daha önce container teknolojisiyle hiç çalışmamış, Linux ve terminal konusunda temel bilgiye sahip geliştiriciler ve sistem yöneticileri

Süre: 3 Gün | Saatler: 09:30 – 16:30

Docker Swarm hariç tüm temel ve ileri konular işlenecektir.

Kurs Hakkında

Kursun Amacı

Bu eğitim, daha önce Docker veya container teknolojileriyle hiç çalışmamış bir kişiyi, üç gün sonunda kendi uygulamalarını güvenli, optimize ve production'a hazır Docker imajlarına dönüştürebilecek; birden fazla konteyneri Docker Compose ile orkestra edebilecek; ağ, veri, güvenlik ve performans konularında bilinçli kararlar verebilecek seviyeye getirmeyi amaçlar.

Kurs boyunca her kavram önce teorik olarak açıklanacak, ardından terminal üzerinde bizzat çalıştırılacak komutlarla pekiştirilecek ve her modül sonunda mini bir laboratuvar çalışmasıyla tamamlanacaktır. Detay atlanmaz; her komut parametresinin anlamı, hangi durumda neden tercih edildiği ve alternatifleri ele alınır.

Önkoşullar

- Temel Linux komutları (cd, ls, cat, grep, ps, chmod, kullanıcı/grup kavramı).
- Bir terminal (bash veya zsh) kullanabilmek.
- Bir metin editörüyle (VS Code, vim, nano) dosya düzenleyebilmek.
- TCP/IP ve port kavramının çok temel seviyede bilinmesi (ek olarak kurs içinde ağ konuları tekrar edilir).

> ⍰ **Not:** Docker ve container teknolojileri hakkında sıfır bilgiyle kursa başlayabilirsiniz. Tüm kavramlar baştan, somut örneklerle ve analojilerle açıklanacaktır.

Gün Bazında Öğrenim Kazanımları

- **Gün 1:** Konteynerleştirmenin neden ve nasıl yapıldığını kavrar; image, container, registry kavramlarını ayırt eder; temel Docker CLI komutlarını akıcı kullanır.
- **Gün 2:** Kendi uygulamaları için profesyonel Dockerfile yazar; çok aşamalı (multi-stage) build'lerle imaj boyutunu azaltır; volume ve network yönetimini uçtan uca çözer.
- **Gün 3:** Docker Compose ile çoklu servisli bir sistemi tek komutla ayağa kaldırır; güvenlik sertleştirmesi yapar; kaynak limitleri ve log yönetimi uygular; CI/CD akışlarına entegre eder.

Günlük Standart Program

Saat	Aktivite
09:30 – 11:00	1. Modül (90 dk)
11:00 – 11:15	Kahve arası
11:15 – 12:30	2. Modül (75 dk)
12:30 – 13:30	Öğle yemeği
13:30 – 15:00	3. Modül (90 dk)
15:00 – 15:15	Kahve arası

Saat	Aktivite
15:15 – 16:30	4. Modül + Günlük özet & Soru-Cevap (75 dk)

GÜN 1 — Docker'a Giriş ve Temel Kavramlar

İlk gün, containerization felsefesini, Docker ekosistemini ve temel günlük komutları sağlam bir zemine oturtmaya ayrılmıştır. Bu gün sonunda katılımcılar, bir imajın ne olduğunu, bir konteynerin bu imajdan nasıl türetildiğini ve Linux çekirdeğinin hangi özelliklerinin konteynerleri mümkün kıldığını net biçimde açıklayabilir olacaktır.

Modül 1 — Containerization Felsefesi ve Docker Mimarisi (09:30 – 11:00)

1.1 Yazılım Dağıtımının Kısa Tarihi

Docker'ı anlamak için neden ortaya çıktığını bilmek gerekir. Son yirmi yılda yazılım dağıtımı üç büyük evreden geçti:

- **Bare metal (fiziksel sunucu):** Uygulama doğrudan fiziksel sunucuya kuruluyordu. Bir sunucuda tek uygulama çalışır, kaynak israfı yüksektir ve yeni sunucu kurmak günler alır.
- **Sanal Makineler (VM):** Hipervizör (VMware, KVM, Hyper-V) sayesinde aynı donanımda birden çok işletim sistemi çalıştırılabilir hâle geldi. Her VM kendi çekirdeğini, kendi dosya sistemini taşır; bu da her birine yüzlerce MB/GB boyut ve dakikalarca açılış süresi getirir.
- **Konteynerler:** Konteynerler, aynı işletim sistemi çekirdeğini paylaşan ancak birbirinden izole edilmiş kullanıcı uzayı (user space) örnekleridir. Açılış süreleri saniye ile ölçülür; boyutları onlarca MB'a kadar düşer.

1.2 "Benim makinemde çalışıyor" Problemi

Geliştiricinin lokalinde çalışan uygulama, test sunucusunda veya prodüksiyonda bir anda patlar. Sebepleri arasında; farklı işletim sistemi sürümleri, farklı kütüphane sürümleri, eksik ortam değişkenleri, farklı dosya yolları, farklı izinler, farklı zaman dilimleri, farklı locale ayarları sıralanır. Docker, uygulamayı tüm bağımlılıklarıyla birlikte tek bir taşınabilir pakette (image) kapatarak bu problemi köklü biçimde çözer.

1.3 Container Nedir, Nasıl Çalışır?

Bir Linux konteyneri, Linux çekirdeğinin iki temel özelliğine dayanır:

- **Namespaces:** Kaynakları (process ID, ağ, mount noktaları, kullanıcılar, IPC) bir süreç grubundan izole eder. Container'ın içinde ps yaptığınızda host'un süreçlerini göremezsiniz; çünkü farklı bir PID namespace'tedir.
- **Control Groups (cgroups):** Süreç gruplarının tüketebileceği CPU, bellek, I/O gibi kaynakları sınırlandırır ve ölçer.
- **Union/Overlay Filesystem:** Birden çok katmanı üst üste bindirerek tek bir dosya sistemi gibi gösterir. Okunabilir alt katmanlar paylaşılır, yazmalar en üstteki ince yazılabilir katmana düşer (copy-on-write).

> ⍰ **Not:** Konteyner bir VM değildir. Host çekirdeğini paylaşır. Bu yüzden Linux konteyneri ancak Linux çekirdeği üzerinde çalışabilir. Windows'ta Docker Desktop, arka planda küçük bir Linux VM (WSL2 veya Hyper-V) çalıştırarak bu sorunu çözer.

1.4 VM vs Container Karşılaştırması

Kriter	Sanal Makine	Konteyner
Çekirdek	Her VM kendi çekirdeğini taşır	Host çekirdeği paylaşılır
Başlatma süresi	Dakikalar	Saniyeler (hatta ms)
Boyut	GB'larca	MB'lar
İzolasyon	Güçlü (donanım seviyesi)	Process seviyesi (daha zayıf ama yeterli)
Performans	Hipervizör katmanı sebebiyle overhead	Native'e çok yakın
Kullanım	Farklı OS'lar, güçlü güvenlik	Mikroservis, CI/CD, dev ortamları

1.5 Docker Ekosistemi ve Mimarisi

Docker bir ürün ailesidir; çoğu zaman "Docker" dendiğinde aslında Docker Engine kastedilir. Ana bileşenler:

- **Docker Engine:** Konteynerleri yöneten core sunucu. Üç parçadan oluşur: dockerd (daemon), REST API, docker (CLI).
- **Docker CLI (docker komutu):** Terminalde yazdığımız komutlar bu istemci aracılığıyla daemon'a gönderilir.
- **containerd:** dockerd'nin altında çalışan, aslında konteynerleri çalıştıran low-level runtime. CNCF projesi.
- **runc:** containerd'nin kullandığı OCI uyumlu en alt seviye runtime. Gerçek namespace ve cgroup çağrılarını yapar.
- **Docker Desktop:** macOS ve Windows için Docker Engine'i küçük bir VM içinde koşturan ve bir arayüz sunan araç.
- **Docker Hub:** Docker'ın resmi public image registry'si (hub.docker.com).
- **Docker Compose:** Birden fazla konteyneri YAML dosyasıyla tanımlayıp yöneten araç.

1.6 OCI: Open Container Initiative

OCI; image formatını, runtime davranışını ve distribution (registry) protokolünü standartlaştıran bir kuruluştur. Bu sayede Docker dışında podman, containerd, CRI-O gibi araçlar aynı imajı çalıştırabilir. Yani bir Docker imajı aslında bir OCI imajıdır ve Docker olmadan da kullanılabilir.

1.7 Docker Alternatifleri

- **Podman:** Red Hat'in daemonsuz, rootless alternatifi. CLI söz dizimi Docker ile neredeyse aynıdır.
- **containerd + nerdctl:** Daha hafif, yalnızca runtime işi yapan sistem.
- **LXC/LXD:** Daha düşük seviyeli, sistem konteynerleri için kullanılır (tam bir Linux dağıtımı içerebilir).
- **Kubernetes (kubelet + CRI):** Üretim ortamlarında orkestra işi için kullanılır; kendisi konteyner motoru değildir ama altında containerd/CRI-O çağırır.

1.8 Kullanım Senaryoları

- Geliştirme ortamlarının tek komutla kurulması (örn. PostgreSQL'i lokale kurmadan çalıştırmak).
- CI/CD pipeline'larında tekrarlanabilir build ve test ortamları.
- Mikroservis mimarisinin her servisini ayrı konteynerde çalıştırmak.
- Legacy uygulamaları paketleyip modernize etmek ("lift and shift").
- Batch işleri, cron görevleri, tek seferlik veri dönüştürme işleri için tek seferlik konteyner.
- Kubernetes cluster'larına temel oluşturmak.

Modül 2 — Kurulum ve İlk Container Çalıştırma (11:15 – 12:30)

2.1 İşletim Sistemine Göre Kurulum

Linux (Ubuntu/Debian)

```
# Eski sürümleri kaldır
sudo apt remove docker docker-engine docker.io containerd runc

# Gerekli paketleri kur
sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release

# Docker GPG anahtarını ekle
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Repository ekle
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

macOS ve Windows

- docker.com/products/docker-desktop adresinden Docker Desktop indirilir ve kurulur.
- Windows'ta WSL2 önerilir; BIOS'tan sanallaştırma aktif olmalıdır.
- macOS'ta Apple Silicon (M serisi) için arm64 paketi, Intel için amd64 paketi indirilir.

2.2 Kurulum Sonrası Ayarlar

```
# Kendi kullanıcınızı docker grubuna ekleyin (sudo'suz komut çalıştırmak için)
sudo groupadd docker        # zaten varsa hata verebilir, önemli değil
sudo usermod -aG docker $USER
newgrp docker               # oturumu yeniden açmadan grubu aktifleştirir

# Docker servisini başlat ve boot'ta otomatik başlamasını sağla (systemd'li sistemler)
sudo systemctl enable --now docker
```

⚠ **Dikkat:** docker grubuna kullanıcı eklemek, o kullanıcıya fiilen root yetkisi verir. Docker daemon root olarak çalıştığı için docker komutu çalıştırabilen biri host dosya sisteminde her şeyi

görebilir/değiştirebilir. Üretim sunucularında bunu bilinçli yapın; alternatif olarak rootless Docker veya Podman düşünün.

2.3 Kurulumu Doğrulama

```
docker --version          # sadece sürüm
docker version            # istemci ve sunucu sürüm detayı
docker info               # sistem-wide kurulum bilgisi (storage driver, cgroup sürümü, vs.)
docker system info        # aynı bilgi; alternatif komut
```

2.4 İlk Konteyneriniz: hello-world

```
docker run hello-world
```

Bu komut çalıştığında arka planda şunlar olur:

1. Docker CLI, UNIX socket (/var/run/docker.sock) üzerinden daemon'a istek gönderir.
2. Daemon, hello-world imajını lokalde arar. Bulamazsa Docker Hub'dan (docker.io/library/hello-world) indirir (pull).
3. Bu imajdan yeni bir konteyner oluşturur (create).
4. Konteyneri başlatır (start). Konteyner içindeki ufak bir program "Hello from Docker!" mesajını STDOUT'a yazar.
5. Docker CLI bu çıktıyı bize gösterir, konteyner görevini tamamlayıp exit yapar.

2.5 docker run'ı Anlamak: En Önemli Flag'ler

docker run komutu, imajdan konteyner oluşturup çalıştıran en kritik komuttur. Sık kullanılan flag'ler:

Flag	Anlamı	Örnek
-d	Detached mode – arka planda çalıştır	docker run -d nginx
-it	İnteraktif + TTY bağlama	docker run -it ubuntu bash
--name	Konteynere isim ver	--name web nginx
-p	Port yayınla: host:container	-p 8080:80
-P	EXPOSE'lu tüm portları rastgele yayınla	docker run -P nginx
-e	Ortam değişkeni geç	-e DB_HOST=db
--env-file	.env dosyasından değişken yükle	--env-file .env
-v	Volume veya bind mount	-v mydata:/data
--rm	Konteyner bitince sil	docker run --rm alpine echo hi
--restart	Restart politikası	--restart unless-stopped
-w	Çalışma dizini ata	-w /app
-u	Kullanıcı/UID olarak çalış	-u 1000:1000
--network	Hangi ağa bağlansın	--network my-net

2.6 Uygulama: Nginx Web Sunucusunu Çalıştırma

```
# nginx imajını çek ve 8080 portundan yayınla
```

```
docker run -d --name web -p 8080:80 nginx:1.27-alpine

# Çalışıp çalışmadığını kontrol et
docker ps

# Tarayıcıdan veya curl ile test
curl http://localhost:8080

# Logları izle
docker logs -f web

# Konteyneri durdur ve sil
docker stop web
docker rm web
```

2.7 Temel Konteyner Yönetim Komutları

```
docker ps                  # çalışan konteynerler
docker ps -a               # tümü (durmuş olanlar dahil)
docker ps -q               # yalnızca ID'ler (script'te handy)
docker stop <name|id>      # nazikçe durdur (SIGTERM, sonra SIGKILL)
docker kill <name|id>      # SIGKILL ile anında öldür
docker start <name|id>     # durmuş konteyneri tekrar başlat
docker restart <name|id>   # durdurup yeniden başlat
docker pause / docker unpause# süreçleri dondur / devam ettir (SIGSTOP/SIGCONT)
docker rm <name|id>        # konteyneri sil (durmuş olmalı)
docker rm -f <name|id>     # çalışıyor olsa bile durdurup sil
docker rename <eski> <yeni>  # konteyner adını değiştir
```

Not: docker run komutu her çağrıldığında YENİ bir konteyner üretir. Aynı konteyneri yeniden çalıştırmak istiyorsanız docker start kullanılmalıdır. Bu ayrımı netleştirmek, başlangıçta en sık yapılan hatalardan birini önler.

Modül 3 — Image'lar, Katmanlar ve Registry'ler (13:30 – 15:00)

3.1 Image Nedir?

Image; bir uygulamayı ve onu çalıştırmak için gereken her şeyi (dosya sistemi, bağımlılıklar, konfigürasyon) içeren, değişmez (immutable) bir şablondur. Bir imajdan istediğiniz kadar konteyner üretebilirsiniz; her biri kendi yazılabilir katmanına sahip olur ama okunabilir katmanlar paylaşılır. Bu, disk ve bellek açısından çok verimlidir.

3.2 Image – Container İlişkisi

Analojik olarak bir imaj, nesne yönelimli programlamadaki "sınıf" gibidir; konteyner ise bu sınıftan üretilmiş bir "nesne" (instance). Sınıf değişmez; nesneler bağımsız olarak yaşar, kendi durumunu taşır ve yok olur.

3.3 Katman (Layer) Kavramı

Docker imajları, birbirinin üstüne yığılmış katmanlardan (layer) oluşur. Her katman, önceki katmanın üstüne yapılan bir değişiklik setidir. Overlay filesystem (overlay2), bu katmanları tek bir birleşik dosya sistemi gibi sunar.

Avantajları:

- Aynı base image'ı kullanan 50 farklı imaj aynı alt katmanları paylaşır – disk tasarrufu.
- Build sırasında değişmeyen katmanlar cache'den kullanılır – hızlı build.
- Push/pull sırasında yalnızca değişen katmanlar indirilir/gönderilir – ağ tasarrufu.

3.4 Image Naming: Registry/Repository:Tag

Tam imaj adı şu şekildedir:

```
<registry>/<namespace>/<repository>:<tag>@<digest>
```

Örnekler:

- nginx → docker.io/library/nginx:latest (varsayılanlar doldurulur)
- ghcr.io/anthropic/app:1.2.3
- myregistry.local:5000/team/api:v2
- nginx@sha256:9c7f... (digest ile — tag'den bağımsız, değişmez referans)

3.5 Tag vs Digest: Immutability Meselesi

Tag değişebilir. Bugün nginx:latest = v1.27.0 iken yarın v1.28.0 olabilir. Eğer production'da tam olarak aynı imajın kullanıldığından emin olmak istiyorsanız tag yerine digest referans vermek daha güvenlidir. Digest, imajın SHA256 özetidir ve asla değişmez.

3.6 Image Komutları

```
docker pull nginx:1.27-alpine       # imaj indir (yoksa)
docker images                       # lokaldeki imajlar
docker image ls --digests           # digest'lerle birlikte listele
docker image inspect nginx          # JSON formatında tüm detay
docker history nginx                # katmanların listesi
docker rmi nginx:1.27-alpine        # tek bir imajı sil
docker image prune                  # dangling (etiketsiz) imajları temizle
docker image prune -a               # kullanılmayan TÜM imajları temizle

# Tag ekleme (mevcut imaja yeni isim):
docker tag nginx:1.27-alpine myrepo/web:v1

# Offline taşıma:
docker save -o nginx.tar nginx:1.27-alpine   # imajı tar'a çıkar
docker load -i nginx.tar                     # başka makinede geri yükle
```

3.7 docker export/import vs save/load

Komut Çifti	Girdi	Çıktı	Katmanları Korur mu?
save / load	image	image (tarball)	Evet, tüm metadata ve katmanlar
export / import	container (çalışan/durmuş)	flat image (tek katman)	Hayır; tek bir rootfs olarak düzleşir

Not: Üretimde imaj taşımak için neredeyse her zaman save/load kullanılır. export/import, bozuk bir konteynerden kurtarma gibi nadir senaryolar için vardır.

3.8 Registry Kavramı

Registry, Docker imajlarını barındıran ve OCI distribution protokolünü konuşan bir sunucudur. Çeşitleri:

- Docker Hub (hub.docker.com) — public; rate-limit uygulanır.
- GitHub Container Registry (ghcr.io) — GitHub hesaplarıyla entegre.
- Amazon ECR, Google Artifact Registry, Azure ACR — bulut sağlayıcıları.
- Harbor, JFrog Artifactory, Nexus — kurum içi self-hosted.
- registry:2 (Docker'ın kendi reference implementasyonu) — lokal test için kullanılabilir.

3.9 Login, Push, Pull

```
# Docker Hub'a giriş
docker login                    # user/pass sorar
docker login ghcr.io            # başka registry
docker login -u kullanici -p $TOKEN   # CI için (güvenli değil; stdin tercih edilir)
echo $TOKEN | docker login -u kullanici --password-stdin

# Tag et ve push'la
docker tag myapp:dev ghcr.io/kullanici/myapp:1.0
docker push ghcr.io/kullanici/myapp:1.0

# Başka makineden çek
docker pull ghcr.io/kullanici/myapp:1.0
```

Not: docker login komutu kimlik bilgilerini ~/.docker/config.json dosyasına şifrelenmemiş olarak yazar (credential helper kurulu değilse). Paylaşımlı sunucularda dikkatli olun; credential-helper (docker-credential-pass, osxkeychain, vs.) kullanımı önerilir.

Modül 4 — Container Yaşam Döngüsü ve İnceleme Araçları (15:15 – 16:30)

4.1 Container Durumları (States)

Durum	Açıklama
created	Oluşturulmuş ama henüz başlatılmamış (docker create).
running	Süreç aktif şekilde çalışıyor.
paused	Süreçler SIGSTOP ile dondurulduğu (docker pause).
restarting	Restart politikası gereği yeniden başlatılıyor.
exited	Süreç tamamlanmış veya çökmüş; exit code taşır.
dead	Daemon düzgün temizleyememiş, arızalı durum.

4.2 Restart Policies (Yeniden Başlatma Politikaları)

- **no** (varsayılan): Asla otomatik başlatma.
- **on-failure[:max]:** Sadece non-zero exit code ile çıktıysa, max kez yeniden başlat.
- **always:** Hangi sebeple durursa dursun yeniden başlat; daemon yeniden açıldığında da başlar.
- **unless-stopped:** always gibi, ancak kullanıcı manuel olarak docker stop ettiyse daemon yeniden açıldığında başlatmaz.

```
docker run -d --restart unless-stopped --name api my/api:1.0
```

4.3 Çalışan Konteynere Girmek: docker exec

```
docker exec -it web bash        # web konteynerinin içinde bash aç
docker exec -it web sh          # alpine gibi minimal imajlarda bash yoktur
docker exec web ls -la /etc     # tek seferlik komut
docker exec -u root web whoami  # root olarak çalıştır
docker exec -e DEBUG=1 web cmd  # ek env var geç
```

⚠ **Dikkat:** docker attach, docker exec ile karıştırılmamalı. attach, konteynerin PID 1 sürecinin STDIN/STDOUT'una bağlanır; çıkınca konteyner de ölebilir. Neredeyse her zaman exec tercih edilir.

4.4 Logları Okumak: docker logs

```
docker logs web                 # tüm log
docker logs -f web              # canlı takip (follow)
docker logs --tail 50 web       # son 50 satır
docker logs --since 10m web     # son 10 dakika
docker logs --until 2026-04-20T12:00 web
docker logs -t web              # her satıra timestamp ekle
```

ℹ **Not:** Docker varsayılan olarak konteynerin STDOUT ve STDERR çıktısını json-file log driver ile host'ta /var/lib/docker/containers/<id>/<id>-json.log dosyasına yazar. Uygulamanız loglarını dosyaya değil terminale (stdout/stderr) basmalıdır — bu 12-factor app prensibinin gereğidir.

4.5 Derin İnceleme: docker inspect

```
docker inspect web                              # tüm JSON
docker inspect --format '{{.State.Status}}' web
docker inspect --format '{{.NetworkSettings.IPAddress}}' web
docker inspect --format '{{json .Mounts}}' web | jq  # mount'ları güzel biçimde
```

4.6 Kaynak Kullanımı: docker stats ve top

```
docker stats            # tüm konteynerlerin canlı CPU/RAM/IO tablosu
docker stats web db     # sadece belirli konteynerler
docker stats --no-stream  # tek çekim, script'te kullanım için
docker top web          # konteyner içindeki süreçler (host PID'leriyle)
```

4.7 Dosya Kopyalama ve Değişiklik İzleme

```
docker cp ./config.yml web:/etc/app/config.yml   # host → container
docker cp web:/var/log/app.log ./app.log         # container → host
docker diff web                                  # rootfs'teki değişiklikler (A:eklendi, C:değişti,
D:silindi)
```

4.8 docker commit: Çalışan Konteynerden İmaj Üretmek

```
docker exec -it web bash
# ...içeride değişiklikler...
exit
docker commit -m "nginx config değişti" web myweb:hotfix
```

⚠ **Dikkat:** docker commit imperative bir işlemdir; hangi değişikliklerin yapıldığını kaydetmez. Üretim için mutlaka Dockerfile (declarative) kullanın; commit sadece acil durum debug'ı için vardır.

4.9 Sistem Temizliği

```
docker system df                # disk kullanım tablosu
```

```
docker system df -v            # detaylı (her imaj ve konteyner için)
docker container prune         # durmuş konteynerleri sil
docker image prune             # dangling image'ları sil
docker image prune -a          # kullanılmayan TÜM imajları sil
docker volume prune            # hiçbir konteynerin kullanmadığı volume'leri sil
docker network prune           # kullanılmayan ağları sil
docker system prune            # tümü birden (container+image+network)
docker system prune -a --volumes  # +volume'ler dahil tam temizlik (!dikkat)
```

⚠ **Dikkat:** docker system prune -a --volumes komutu geri alınamaz. Ortak kullanılan sunucularda mutlaka onaylatın; verileri volume'de tutan bir DB konteyneri silinmemiş olsa bile volume'u yanlışlıkla silebilirsiniz.

4.10 Günün Labı: Nginx ile Konteyner Yaşam Döngüsü

6. docker run -d --name web -p 8080:80 nginx:1.27-alpine ile bir nginx çalıştırın.
7. docker exec -it web sh ile içeri girin, /etc/nginx/nginx.conf dosyasını inceleyin.
8. docker cp ile host'tan hazırladığınız özel bir index.html dosyasını /usr/share/nginx/html/ dizinine kopyalayın.
9. Tarayıcıdan http://localhost:8080 adresini kontrol edin, değişikliği görün.
10. docker diff web ile hangi dosyaların değiştiğini listeleyin.
11. docker stats --no-stream ile RAM/CPU kullanımını görün.
12. docker logs -f web'i bir pencerede açık tutup diğer pencereden birkaç kez curl atın, log'ların akışını izleyin.
13. docker stop web && docker rm web ile temizlik yapın.

GÜN 2 — Dockerfile, Veri Yönetimi ve Ağ

İkinci gün, tüketici pozisyonundan üretici pozisyonuna geçişi temsil eder. Kendi imajlarımızı yazacak, bu imajları optimize edecek, konteynerlerimizin verilerini kalıcı hâle getirecek ve konteynerler arası ağ haberleşmesini baştan sona çözeceğiz.

Modül 5 — Dockerfile Temelleri ve Talimatlar (09:30 – 11:00)

5.1 Neden Dockerfile?

İmaj oluşturmanın iki yolu vardır: (1) docker commit ile manuel, (2) Dockerfile ile declarative (bildirimsel). Dockerfile bir reçetedir — metin dosyasıdır, git'e girer, code review edilebilir, reproducible'dır. Üretim için tek doğru yol Dockerfile'dır.

5.2 Minimal Bir Dockerfile

```
# Base image seç
FROM node:20-alpine

# Konteyner içi çalışma dizini
WORKDIR /app

# Önce bağımlılık listesi (cache'i iyi kullanmak için ayrı)
COPY package*.json ./
RUN npm ci --omit=dev

# Ardından kod
COPY . .

# Konteyner açılınca çalışacak komut
CMD ["node", "server.js"]
```

5.3 Build Context ve docker build

```
# Dockerfile'ın bulunduğu dizinde:
docker build -t myapp:1.0 .

# Alternatifler:
docker build -f Dockerfile.prod -t myapp:prod .
docker build -t myapp:1.0 --no-cache .   # cache'i bypass et
docker build -t myapp:1.0 --progress=plain .  # tüm çıktıyı göster (CI'da handy)
```

Komutun sonundaki . (nokta) build context'tir: Bu dizin tarball haline getirilip daemon'a gönderilir. COPY/ADD talimatları bu context içinden kaynak alır.

> ⚠ **Dikkat:** Build context'i küçük tutun. Büyük bir dizinde build yaparsanız node_modules, .git, test verisi gibi yüzlerce MB veri gereksiz yere daemon'a yollanır ve build hem yavaşlar hem de nihai imaj şişebilir. Bunun için .dockerignore kullanın.

5.4 .dockerignore

```
# .dockerignore
node_modules
.git
.gitignore
```

```
*.log
Dockerfile*
.env
tests/
coverage/
*.md
```

5.5 Dockerfile Talimatları — Tek Tek İnceleme

FROM — Base Image

Her Dockerfile FROM ile başlamalıdır (ARG ile yalnızca FROM'dan önce tanımlanabilir). Base image seçimi nihai imaj boyutunu, güvenliğini ve hızını etkiler.

- nginx, python, node gibi "official" imajlar Debian tabanlıdır, büyüktür ama uyumludur.
- -alpine sürümleri musl libc tabanlıdır, çok küçüktür ama bazı C kütüphaneleriyle uyum sorunları olabilir.
- -slim sürümleri Debian minimum kurulumdur; alpine'dan uyumsuzluksuz, full'dan küçüktür.
- distroless (gcr.io/distroless/...) imajları shell dahi içermez; en güvenli ama debug'ı zor.
- scratch: boş imaj; yalnızca statik binary deploy ederken kullanılır.

RUN — Build Sırasında Komut Çalıştır

```
# Shell form (sh -c ile çalışır):
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

# Exec form (tavsiye edilen, ayrıştırıcı yok):
RUN ["apt-get", "install", "-y", "curl"]
```

RUN her çağrıldığında yeni bir katman oluşturur. Bu yüzden paket kurulumu ve cache temizliği aynı RUN içinde olmalıdır, aksi halde silinen dosyalar önceki katmanda kalır ve imaj küçülmez.

CMD vs ENTRYPOINT — En Çok Karıştırılan İkili

İkisi de konteyner başladığında ne çalışacağını belirler, ama farklı rollerdedir:

- **ENTRYPOINT:** Konteynerin "amacı". docker run sonrası verilen argümanlar bunun arkasına eklenir.
- **CMD:** Varsayılan argümanlar. docker run sonrası argüman verilirse ezilir.

```
# Yalnızca CMD
CMD ["python", "app.py"]
# docker run myapp  →  python app.py
# docker run myapp --debug  →  --debug   (CMD tamamen ezildi!)

# Yalnızca ENTRYPOINT
ENTRYPOINT ["python", "app.py"]
# docker run myapp  →  python app.py
# docker run myapp --debug  →  python app.py --debug

# İkisi birlikte (en güçlü kalıp)
ENTRYPOINT ["python", "app.py"]
CMD ["--port=8000"]
# docker run myapp  →  python app.py --port=8000
# docker run myapp --port=9000  →  python app.py --port=9000
```

ℹ **Not:** Her ikisinin de "exec form" (JSON array) tercih edilmelidir. Shell form (tek string) sh -c ile çalıştırır ve PID 1 shell olur; bu da sinyallerin uygulamaya iletilmemesine (graceful shutdown sorunu) neden olur.

COPY vs ADD

COPY ve ADD dosya kopyalar ama farkları vardır:

- COPY sadece lokal dosyaları kopyalar. Basit ve öngörülebilir.
- ADD ayrıca uzak URL'den indirebilir ve tar.gz arşivleri otomatik açabilir.
- Best practice: URL veya otomatik açma gerekmiyorsa her zaman COPY kullanın.

WORKDIR

Çalışma dizinini ayarlar, yoksa oluşturur. cd ile karıştırılmamalıdır; cd RUN içinde shell'e özeldir ve sonraki talimatlara geçmez.

USER

Hangi kullanıcıyla çalışılacağını belirler. Varsayılan root'tur; üretimde kesinlikle non-root kullanıcıya düşülmelidir:

```
RUN addgroup -S app && adduser -S -G app app
USER app
```

ENV ve ARG

- **ENV:** Hem build sırasında hem konteyner çalışırken mevcut. Konteyner metadata'sına yazılır.
- **ARG:** Yalnızca build zamanı. --build-arg ile dışarıdan değer alabilir. Konteyner içinde görünmez.

```
ARG NODE_VERSION=20
FROM node:${NODE_VERSION}-alpine

ENV APP_PORT=3000 \
   NODE_ENV=production
```

⚠ **Dikkat:** Hassas bilgiyi (API key, şifre) ne ARG ne de ENV ile imaja gömün. Bunlar imaj metadata'sına yazılır ve docker history ile görünür. BuildKit secret mount veya runtime env/secret kullanın.

EXPOSE

Belgeleme amaçlıdır; "bu imaj bu portu dinliyor" der. Aslında port yayınlamaz; yayınlama -p veya -P ile yapılır.

LABEL

İmaja metadata ekler. Otomatikleşmiş sistemlerde filtreleme için yararlıdır:

```
LABEL org.opencontainers.image.authors="team@company.com"
LABEL org.opencontainers.image.source="https://github.com/ornek/repo"
LABEL org.opencontainers.image.version="1.0.0"
```

VOLUME

Bir dizini volume olarak işaretler; konteyner silinse bile Docker o dizine anonim bir volume bağlar. DB imajlarında sık kullanılır (postgres imajı /var/lib/postgresql/data için VOLUME belirtir).

HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost/ || exit 1
```

docker ps çıktısında STATUS sütununda healthy/unhealthy olarak görünür. Compose depends_on condition: service_healthy ile birlikte kritik rol oynar.

ONBUILD, SHELL, STOPSIGNAL

- **ONBUILD:** Bu imaj başka bir imajda FROM olarak kullanıldığında çalışacak talimat. Base imaj yazarları için.
- **SHELL:** RUN'ın shell form'unda hangi shell'i kullanacağını değiştirir. Windows konteynerlerinde PowerShell için kullanılır.
- **STOPSIGNAL:** docker stop'ta gönderilecek sinyali belirler (varsayılan SIGTERM). Bazı süreçler SIGQUIT ile daha iyi kapanır (nginx gibi).

5.6 Layer Caching'i Doğru Kullanmak

Docker her talimat için bir katman oluşturur ve girdiler (komut metni ve COPY'lenen dosyaların hash'i) değişmediyse cache'den gelir. Hızlı build'ler için kural: Değişme ihtimali en düşük şeyi en üste koy.

```
# KÖTÜ — her kod değişikliğinde npm install baştan çalışır
COPY . .
RUN npm ci

# İYİ — kod değişse de paket listesi değişmediyse cache'den gelir
COPY package*.json ./
RUN npm ci
COPY . .
```

5.7 Uygulama: Basit Node.js API için Dockerfile

```
FROM node:20-alpine
WORKDIR /app

ENV NODE_ENV=production \
   PORT=3000

COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY . .

RUN addgroup -S app && adduser -S -G app app && chown -R app:app /app
USER app

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health
|| exit 1
CMD ["node", "server.js"]
```

Modül 6 — Multi-Stage Build ve İmaj Optimizasyonu (11:15 – 12:30)

6.1 Multi-Stage Build Nedir?

Bir Dockerfile içinde birden çok FROM tanımlayarak farklı aşamalar oluşturmaktır. Derleme için gerekli büyük araçlar build aşamasında kalır; yalnızca son çıktı (binary, derlenmiş statik dosyalar) runtime aşamasına kopyalanır. Sonuç: onlarca kat daha küçük ve güvenli imajlar.

6.2 Go Uygulaması İçin Örnek

```
# Aşama 1: build
FROM golang:1.22-alpine AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -ldflags='-s -w' -o /out/app ./cmd/app

# Aşama 2: runtime
FROM gcr.io/distroless/static:nonroot
COPY --from=builder /out/app /app
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

Build aşamasındaki 300+ MB Go toolchain ve kaynak kod nihai imaja gitmez; son imaj yalnızca yürütülebilir binary ve minimum işletim sistemi içerir (~10 MB).

6.3 Node.js İçin Multi-Stage

```
# deps
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

# build (TS → JS)
FROM deps AS build
COPY . .
RUN npm run build && npm prune --production

# runtime
FROM node:20-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY --from=build /app/package.json ./
USER node
CMD ["node", "dist/server.js"]
```

6.4 Belirli Bir Aşamayı Build Etme

```
docker build --target=build -t myapp:debug .
```

CI'da test amaçlı yalnızca build aşamasını çekmek istediğinizde kullanışlıdır.

6.5 BuildKit: Modern Build Engine

Docker 23+'da default olarak aktiftir. Öncekine göre paralel build, önbellek mount'ı, secret mount ve SSH forwarding gibi kritik yetenekler getirir.

```
# Eski sistemlerde aktifleştirmek için:
export DOCKER_BUILDKIT=1
```

6.6 Cache Mount: Paket Cache'ini Kalıcı Tutmak

```
# syntax=docker/dockerfile:1.7
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
COPY . .
```

Build'ler arası pip cache'i saklanır; imaj boyutu artmaz çünkü cache imaja yazılmaz.

6.7 Secret Mount: Şifreyi İmaja Yazmadan Kullanmak

```
# Dockerfile
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) npm ci

# Build komutu
docker build --secret id=npm_token,src=$HOME/.npmrc -t myapp .
```

6.8 Multi-Platform Build (buildx)

```
# Bir kez builder oluştur
docker buildx create --name multi --use
docker buildx inspect --bootstrap

# Aynı anda amd64 ve arm64 için build et ve push'la
docker buildx build --platform linux/amd64,linux/arm64 \
  -t ghcr.io/kullanici/app:1.0 --push .
```

Not: Apple Silicon bir makinede geliştirip Linux x86_64 sunucuya deploy ederken buildx kritik önemdedir; aksi halde yanlış mimari imaj ile karşılaşılır.

6.9 İmaj Boyutu Optimizasyon Kontrol Listesi

- Base image olarak mümkünse alpine, slim veya distroless seçin.
- Multi-stage build ile derleme araçlarını runtime'a taşımayın.
- apt/apk/yum cache'ini aynı RUN içinde temizleyin.
- Gereksiz dosyaları .dockerignore'a koyun.
- Tek bir büyük RUN yerine mantıklı sayıda katman kullanın (cache efekti için); ama lüzumsuz katman da açmayın.
- docker history ve dive (3. parti araç) ile katmanları analiz edin.

Modül 7 — Volume, Bind Mount ve Veri Kalıcılığı (13:30 – 15:00)

7.1 Konteyner İçinde Veri Neden Kaybolur?

Konteyner silindiğinde (docker rm), onun writable layer'ı (en üstteki ince katman) da silinir. Yani konteyner içinde oluşturulan dosyalar, açıkça bir volume veya bind mount'a yazılmadıkça konteyner ömrüyle sınırlıdır. Bu durum özellikle veritabanları için kritik felakete yol açar.

7.2 Üç Çeşit Mount

Tür	Yönetim	Host Yolu	Kullanım
Volume	Docker yönetir (/var/lib/docker/volumes)	Gizli, Docker tarafından yönetilir	Prod DB'ler, paylaşılan state
Bind mount	Siz belirtirsiniz	Tam host yolu verilir (/home/x/proj:/app)	Dev ortamı, config dosyaları
tmpfs mount	Bellek	-	Geçici dosyalar, şifre, kısa ömürlü cache

7.3 Volume Komutları

```
docker volume create mydata
docker volume ls
docker volume inspect mydata
docker volume rm mydata
docker volume prune          # kullanılmayan tüm volume'leri sil
```

7.4 -v ve --mount Syntax'ı

```
# -v syntax (daha kısa, eski):
docker run -d -v mydata:/var/lib/mysql mysql:8
docker run -d -v /host/path:/container/path:ro nginx
docker run -d -v /host/path:/container/path:rw,Z nginx  # SELinux label

# --mount syntax (daha açık, önerilen):
docker run -d --mount type=volume,source=mydata,target=/var/lib/mysql mysql:8
docker run -d --mount type=bind,source=/host/path,target=/app,readonly nginx
docker run -d --mount type=tmpfs,target=/tmp,tmpfs-size=64m alpine
```

Not: --mount syntax'ında tüm parametreler isimlendirilmiştir; hata yaptığınızda Docker net mesaj verir. -v'de yazım hatası yaparsanız Docker sessizce yeni bir anonim volume oluşturur ve hata anlaşılmaz. Yeni kodda --mount tercih edin.

7.5 Named vs Anonymous Volume

- Named: docker volume create ile veya -v isim:/path ile oluşan, ismi olan, kalıcı volume.
- Anonymous: -v /path ile (host path vermeden) oluşan, rastgele hash isimli volume. Konteyner silinip --rm flag'i kullanıldığında veya docker volume prune ile silinebilir; takibi zordur.

7.6 Dockerfile'daki VOLUME Talimatı

```
VOLUME /var/lib/mysql
```

Bu talimat, konteyner çalışınca Docker'ın /var/lib/mysql için otomatik olarak anonim volume oluşturmasını sağlar. Kullanıcı -v ile kendi named volume'ünü bağlarsa elbette bu geçerli olur. Resmi DB imajları (mysql, postgres, mongo) bunu kullanır.

7.7 Read-Only Mount

```
docker run -d -v appconfig:/etc/app:ro myapp
```

Konfigürasyon dosyalarını konteynere salt-okunur bağlamak, saldırgan içeri girse bile config'i değiştirmesini zorlaştırır.

7.8 Volume Backup/Restore

```
# mydata volume'ünü tar'a al
docker run --rm \
  -v mydata:/data:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/mydata-backup.tar.gz -C /data .

# Aynı volume'e geri yükle
docker run --rm \
  -v mydata:/data \
  -v $(pwd):/backup \
  alpine sh -c 'cd /data && tar xzf /backup/mydata-backup.tar.gz'
```

7.9 Permission ve UID/GID Problemleri

Bind mount ile host'taki bir dizini konteynere bağladığınızda, host'taki dosya sahibi UID/GID değişmez. Konteyner içindeki kullanıcının UID'si farklıysa "Permission denied" alırsınız. Çözümler:

- docker run -u $(id -u):$(id -g) ile host kullanıcı UID/GID'siyle çalıştırın.
- Dockerfile'da app kullanıcısının UID'sini host ile eşleştirin.
- Volume kullanın (Docker UID meselesini çözer).

7.10 Volume Driver Kavramı

Varsayılan driver local'dir; ancak NFS, CIFS (SMB), AWS EBS, Portworx gibi driver'larla ağ üstünde paylaşılan veya bulut depolamaya yazan volume'ler tanımlanabilir:

```
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=10.0.0.5,rw \
  --opt device=:/exports/data \
  nfsvol
```

7.11 Lab: PostgreSQL Veri Kalıcılığı

```
# 1) Volume oluştur
docker volume create pgdata

# 2) Postgres'i çalıştır
docker run -d --name pg \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 postgres:16-alpine

# 3) İçeri gir, DB oluştur, tablo ekle
docker exec -it pg psql -U postgres -c 'CREATE DATABASE mydb;'

# 4) Konteyneri sil
docker rm -f pg

# 5) Aynı volume'ü yeni konteynere bağla; DB hâlâ burada!
docker run -d --name pg2 \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 postgres:16-alpine
```

```
docker exec -it pg2 psql -U postgres -c 'SELECT datname FROM pg_database;'
```

Modül 8 — Docker Networking (15:15 – 16:30)

8.1 Network Model'inin Temelleri

Docker her konteynere kendi network namespace'ini verir: kendi ağ arayüzü, IP'si, routing tablosu, iptables kuralları. Konteynerleri bağlamak için Docker çeşitli driver'lar sunar.

8.2 Yerleşik Network Driver'lar

Driver	Kullanım
bridge	Varsayılan; aynı host'ta konteynerlerin birbirini görmesi için özel ağ oluşturur.
host	Konteyner, host'un network namespace'ini paylaşır; izolasyon yoktur, performans en yüksek seviyededir.
none	Konteynerin hiçbir ağı olmaz; tam izolasyon için.
macvlan	Konteynere fiziksel ağdan kendi MAC + IP'si verilir; legacy uygulamalar için.
ipvlan	macvlan'e benzer ama aynı MAC'i paylaşır; ağ donanımı kısıtları varsa tercih.
overlay	Swarm/Kubernetes gibi çoklu host ağlarında kullanılır (bu kursta detaya girmiyoruz).

8.3 Default Bridge vs User-Defined Bridge

Docker kurulunca otomatik bir bridge0 / docker0 ağı oluşturur ve isim verilmemiş tüm konteynerleri buraya takar. Ancak bu "default bridge" ağda:

- Konteynerler birbirini İSİM ile ÇÖZEMEZ. Yalnızca IP ile haberleşebilirler.
- Tüm konteynerler aynı ağda olduğu için izolasyon zayıftır.

Oysa kendi oluşturduğunuz bir user-defined bridge'te:

- Docker built-in DNS, konteyner isimlerini IP'ye çözer. docker run --name web ile çalıştırdığınız konteynere, aynı ağdaki başka bir konteynerden ping web yapabilirsiniz.
- Her ağ izole bir broadcast domain'dir; farklı projelerinizin konteynerleri birbirini görmez.

> ⍰ **Not:** Üretim ve hatta geliştirme için her zaman kendi ağınızı oluşturun; default bridge'e asla güvenmeyin. Docker Compose bunu otomatik yapar.

8.4 Network Komutları

```
docker network create mynet
docker network create --driver bridge --subnet 10.10.0.0/24 --gateway 10.10.0.1 mynet2
docker network ls
docker network inspect mynet
docker network rm mynet
docker network prune

# Çalışan bir konteyneri ağa bağla/çıkar
docker network connect mynet web
docker network disconnect mynet web
```

8.5 İsimle Haberleşme: DNS

```
docker network create app-net
docker run -d --name db --network app-net -e POSTGRES_PASSWORD=x postgres:16-alpine
docker run --rm --network app-net alpine ping -c 2 db
# db ismi otomatik olarak db konteynerinin IP'sine çözülür
```

8.6 Port Publishing: -p

```
docker run -d -p 8080:80 nginx          # 8080 (host) → 80 (container)
docker run -d -p 127.0.0.1:8080:80 nginx  # yalnızca localhost'tan dış dünya değil
docker run -d -p 8080:80/tcp -p 8080:80/udp nginx
docker run -d -P nginx                  # EXPOSE'lu tüm portları rastgele yayınla
```

⚠ **Dikkat:** Üretim sunucularında -p 0.0.0.0:... açıyorsanız (default) portu tüm internete açıyor olabilirsiniz. Güvenlik duvarı ve bind adresini bilerek seçin; DB portlarını asla dışa açmayın.

8.7 Network Alias

```
docker run -d --name db --network app-net --network-alias database --network-alias primary postgres:16-alpine
# Artık db, database ve primary isimleri aynı konteynere çözer
```

8.8 Host Network

```
docker run -d --network host nginx
```

Konteyner host'un tüm ağ arayüzlerini doğrudan kullanır; -p gereksizdir. Avantaj: en yüksek performans. Dezavantaj: izolasyon yok, port çakışması kolay, Docker Desktop'ta (macOS/Windows) beklendiği gibi çalışmayabilir.

8.9 None Network

```
docker run -it --network none alpine sh
```

Sıfır ağ. Güvenlik testi veya tamamen offline batch görev için kullanılır.

8.10 Docker'ın iptables Üzerindeki Etkisi

Docker, kurulunca iptables'a pek çok kural ekler (DOCKER zinciri). -p ile yayınladığınız portlar için DNAT kuralları oluşturur. UFW gibi bir güvenlik duvarı çalıştırıyorsanız Docker kuralları UFW'yu atlayabilir; bu bilinen bir "gotcha"dır ve --iptables=false ya da docker-compose ağ ayarlarıyla yönetilmelidir.

8.11 Lab: Web + DB İki Konteyner

```
docker network create app-net

docker run -d --name db --network app-net \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=myapp \
  postgres:16-alpine

docker run -d --name web --network app-net -p 8080:80 \
  -e DATABASE_URL=postgres://postgres:secret@db:5432/myapp \
  nginx:1.27-alpine

# web konteynerinden db'yi adıyla sor
docker exec web sh -c 'apk add --no-cache curl postgresql-client && psql $DATABASE_URL -c "SELECT 1;"'
```

```
# temizlik
docker rm -f web db && docker network rm app-net
```

GÜN 3 — Docker Compose, Güvenlik, Production

Son gün, birden fazla konteyneri bir arada yönetmeyi, production koşullarına uygun biçimde güvenlik, performans ve gözlemlenebilirlik (observability) pratiklerini uygulamaya ayrılmıştır. Gün sonunda katılımcı, gerçek bir ürünü nasıl güvenli şekilde Docker'la deploy edeceğini bilecek hâle gelir.

Modül 9 — Docker Compose Temelleri (09:30 – 11:00)

9.1 Neden Compose?

Bir web + bir DB + bir Redis + bir nginx proxy ayağa kaldırmak için 4 adet docker run komutu, ağ oluşturma, volume oluşturma, doğru bağımlılık sırası... Hepsi manuel ve hataya açık. Docker Compose tüm bu yapılandırmayı tek bir compose.yaml dosyasında bildirim yazar, docker compose up ile tek komutta ayağa kaldırır.

9.2 docker-compose vs docker compose (v1 / v2)

- docker-compose (tire ile) → Python ile yazılmış v1. Deprecated; artık kurulmaz.
- docker compose (boşluk ile) → Go ile yazılmış v2, Docker Engine'in plugin'i. Güncel sürüm.

> ⚠ **Not:** Compose file içinde version: 3.8 gibi satırlar yazmak ARTIK GEREKLİ DEĞİL. V2 tüm modern özellikleri destekler ve version satırını "obsolete" warning'iyle görmezden gelir.

9.3 Minimal compose.yaml

```
services:
  web:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"

# Bulunduğun dizinde çalıştır
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

9.4 Proje İsmi ve İzolasyon

Compose her çağrıda bir "proje" adı türetir (varsayılan: dizinin adı). Bu isim, oluşturduğu konteyner, ağ ve volume'lerin önüne prefix olarak konur: myproject_web_1, myproject_default ağı, myproject_pgdata volume'ü gibi. İki farklı proje aynı anda ayağa kalkabilir ve çakışmazlar.

```
docker compose -p customname up -d
# veya
export COMPOSE_PROJECT_NAME=customname
```

9.5 Bir Servisin Tam Anatomisi

```
services:
  api:
    image: myrepo/api:1.0        # ya image ya build
    build:
      context: ./api
      dockerfile: Dockerfile.prod
```

```
    args:
      NODE_VERSION: 20
  container_name: api          # opsiyonel; kullanırsanız scale edemezsiniz
  restart: unless-stopped
  environment:
    - NODE_ENV=production
    - DB_HOST=db
  env_file:
    - .env
  ports:
    - "3000:3000"
  volumes:
    - ./uploads:/app/uploads
    - logs:/var/log/app
  depends_on:
    db:
      condition: service_healthy
  networks:
    - backend
  healthcheck:
    test: ["CMD", "wget", "-qO-", "http://localhost:3000/health"]
    interval: 30s
    timeout: 3s
    retries: 3
    start_period: 20s
  command: ["node", "dist/server.js"]
  user: "1000:1000"
  read_only: true
  tmpfs:
    - /tmp
```

9.6 depends_on'un İnce Ayrıntısı

Basit depends_on yalnızca konteynerin başlatılma SIRASINI belirler; o servisin gerçekten HAZIR olduğunu garanti etmez. DB konteyneri ayağa kalkmış gibi görünse de içindeki PostgreSQL hâlâ initialize olurken uygulamanız bağlantı deneyip patlayabilir. Çözüm: condition: service_healthy kullanıp bağımlılık hedefinde healthcheck tanımlamak.

9.7 Compose Komutları

```
docker compose up                    # foreground'da ayağa kaldır
docker compose up -d                 # arka planda
docker compose up -d --build         # imajları yeniden build et
docker compose up -d --force-recreate # aynı config'le bile recreate et
docker compose down                  # konteyner ve default ağı sil
docker compose down -v               # ek olarak volume'leri de sil (dikkat!)
docker compose down --remove-orphans  # tanımda olmayanları da sil
docker compose ps                    # bu projeye ait konteynerler
docker compose logs -f [servis]      # logları takip
docker compose exec api sh           # bir servise gir
docker compose run --rm api npm test  # yeni, kısa ömürlü konteynerde komut
docker compose restart api           # tek servisi yeniden başlat
docker compose pull                  # tüm imajları çek
docker compose build api             # yalnızca api'yi build et
docker compose config                # final config'i (variable substitution dahil) yazdır
```

9.8 Uygulama: WordPress + MySQL

```
services:
  db:
    image: mysql:8
    restart: always
    environment:
      MYSQL_DATABASE: wp
      MYSQL_USER: wp
      MYSQL_PASSWORD: wppass
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - dbdata:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5

  wordpress:
    image: wordpress:6-php8.2-apache
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wp
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wp
    volumes:
      - wpdata:/var/www/html

volumes:
  dbdata:
  wpdata:
```

Modül 10 — Compose İleri Seviye Özellikler (11:15 – 12:30)

10.1 .env Dosyası ve Variable Substitution

```
# .env
APP_PORT=8080
DB_PASSWORD=sUpeRs3cret

# compose.yaml
services:
  api:
    image: myapi:1.0
    ports:
      - "${APP_PORT:-3000}:3000"
    environment:
      - DB_PASSWORD=${DB_PASSWORD?DB_PASSWORD boş olamaz}
```

${VAR:-default} yok/boşsa default, ${VAR:?hata} yok/boşsa hata verir, ${VAR-default} yoksa default (boş olmak okey).

10.2 Multiple Compose Files (Override)

```
# Docker Compose otomatik olarak compose.yaml + compose.override.yaml'ı birleştirir
docker compose up -d

# Manuel birden çok dosya:
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

Base dosyada genel tanımlar, override dosyasında ortama özel farklar tutulur (dev için debug flag açık, prod için kapalı vb.).

10.3 Profiles

```
services:
  app:
    image: myapp
  worker:
    image: myworker
    profiles: ["workers"]
  test:
    image: mytester
    profiles: ["tests"]

# docker compose up -d                → yalnızca app
# docker compose --profile workers up  → app + worker
# docker compose --profile tests up    → app + test
```

10.4 Named Networks ve External Network

```
networks:
  frontend:
  backend:
    internal: true          # dış dünyaya çıkamaz
  shared:
    external: true           # başka projeden mevcut ağı kullan

services:
  web:
    networks: [frontend, backend]
  db:
    networks: [backend]       # frontend'den erişilemez
```

⍰ **Not:** internal: true; güvenlik açısından önemlidir. DB gibi servisleri yalnızca internal bir ağda tutup, web/app'i hem internal hem external ağa bağlayarak DB'nin dışa hiç açılmamasını sağlayabilirsiniz.

10.5 Named Volumes ve External Volumes

```
volumes:
  pgdata:
  shared_logs:
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /srv/logs
  existing_vol:
    external: true
```

10.6 Secrets ve Configs

```yaml
secrets:
  db_password:
    file: ./secrets/db_password.txt

configs:
  nginx_conf:
    file: ./nginx.conf

services:
  db:
    image: postgres:16-alpine
    secrets: [db_password]
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
  web:
    image: nginx:1.27-alpine
    configs:
      - source: nginx_conf
        target: /etc/nginx/nginx.conf
```

10.7 Scaling

```
docker compose up -d --scale web=3
# container_name tanımlıysa scale çalışmaz; kaldırın
```

Compose bu üç instance'a yük dağıtımı yapmaz; önüne bir reverse proxy (nginx, Traefik) koymak gerekir. Gerçek yük dengelemesi orkestra katmanının (Kubernetes) işidir.

10.8 Deploy Section — Kaynak Limitleri

```yaml
services:
  api:
    image: myapi:1.0
    deploy:
      resources:
        limits:
          cpus: "1.5"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M
```

Not: Swarm dışında da çalışır; Compose v2, deploy.resources.limits'i docker run --cpus ve --memory olarak uygular. replicas ve endpoint_mode gibi bazı alanlar yalnızca Swarm'da anlamlıdır.

10.9 Anchor'lar ve Extension Fields (YAML hileleri)

```yaml
x-default-logging: &default-logging
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"

services:
  web:
    image: nginx
```

```
    logging: *default-logging
  api:
    image: myapi
    logging: *default-logging
```

10.10 Lab: 4 Servisli Kompleks Stack

```
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80"]
    configs:
      - source: nginx_conf
        target: /etc/nginx/conf.d/default.conf
    depends_on: [api]
    networks: [frontend]

  api:
    build: ./api
    environment:
      - DB_HOST=db
      - REDIS_HOST=cache
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }
    networks: [frontend, backend]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_pw
    secrets: [db_pw]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10
    networks: [backend]

  cache:
    image: redis:7-alpine
    networks: [backend]

networks:
  frontend:
  backend:
    internal: true

volumes:
  pgdata:

secrets:
  db_pw: { file: ./secrets/db_pw.txt }

configs:
```

```
nginx_conf: { file: ./nginx/default.conf }
```

Modül 11 — Güvenlik, Kaynak Yönetimi ve Logging (13:30 – 15:00)

11.1 Container Güvenliğinin Temel İlkeleri

- En az yetki prensibi: Konteyner yalnızca ihtiyacı olan şeyi yapabilmeli.
- Defense-in-depth: Tek bir katmana güvenme; birden çok savunma üst üste.
- Immutable: İmaj kendi içinde bir kez derlenir, değişmez şekilde deploy edilir.
- Saldırı yüzeyini minimize et: Gereksiz paket, gereksiz kullanıcı, gereksiz port yok.

11.2 Konteyneri Non-Root Çalıştırmak

Varsayılanda konteyner içi süreç root olarak çalışır. Eğer bir güvenlik açığı ile container escape gerçekleşirse, host'ta da root olmak daha kolaydır. Bu yüzden:

```
# Dockerfile'da
RUN addgroup -S app && adduser -S -G app app
USER app

# veya runtime'da
docker run -u 1000:1000 myapp
```

11.3 Read-Only Root Filesystem

```
docker run --read-only --tmpfs /tmp myapp
# Compose:
#   read_only: true
#   tmpfs: ["/tmp", "/var/run"]
```

Saldırgan içeri girse bile binary ekleyemez, dosya değiştiremez.

11.4 Linux Capabilities

Root'un yetkileri 30'dan fazla capability'ye ayrılmıştır (NET_ADMIN, SYS_ADMIN, CHOWN, ...). Docker varsayılan olarak bir bolus (kabul edilebilir) alt küme verir. Daha sıkı yapın:

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx
# Compose:
#   cap_drop: [ALL]
#   cap_add: [NET_BIND_SERVICE]
```

11.5 seccomp, AppArmor, SELinux

- seccomp: syscall'ları filtreler. Docker default profili çoğu güvenlik araştırmacısının tavsiyesiyle yazılmıştır; bunu kapatmak (--security-opt seccomp=unconfined) neredeyse her zaman kötü fikirdir.
- AppArmor (Ubuntu/Debian): Docker default bir profil yükler; özel profillerle sıkılaştırma yapılabilir.
- SELinux (RHEL/Fedora): -v ile bind mount'ta :Z veya :z sonekiyle dosyaların SELinux etiketini ayarlamak gerekir.

11.6 --privileged Tuzağı

⚠ **Dikkat:** docker run --privileged vermek, konteyneri host'a yakın ayrıcalıklarla çalıştırır; hemen hemen hiçbir güvenlik katmanı kalmaz. Bu flag'i yalnızca gerçekten zorunlu olduğunda (Docker-in-

Docker, bazı sistem araçları) ve bilinçli kullanın. "Çalışmıyor" diye eklemeyin; muhtemelen spesifik bir capability yetecektir.

11.7 İmaj Tarama

```
# Docker Scout (Docker Desktop ve CLI ile geliyor)
docker scout cves myimage:1.0
docker scout recommendations myimage:1.0

# Trivy (standalone, popular)
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image myimage:1.0

# Grype
grype myimage:1.0
```

11.8 Kaynak Limitleri (cgroups)

Flag	Anlamı
--memory 512m	RAM limiti; aşılırsa konteyner OOM kill edilir.
--memory-swap	RAM + swap toplamı. --memory ile aynı yapılırsa swap kullanılmaz.
--memory-reservation	Yumuşak limit; baskı anında bu değerin altına iner.
--cpus 1.5	1.5 mantıksal CPU çekirdeği kadar.
--cpu-shares 512	Göreli ağırlık (varsayılan 1024).
--cpuset-cpus 0,1	Yalnızca belirli CPU'larda koşsun.
--pids-limit 100	Konteynerin açabileceği maks. PID sayısı; fork-bomb koruması.
--ulimit nofile=1024:2048	Dosya tanıtıcı limiti (soft:hard).

11.9 OOM Killer Davranışı

Memory limit aşıldığında Linux kernel OOM killer devreye girer. Bazı süreçlerde (özellikle DB) bu kritik veri kaybına yol açabilir. --oom-kill-disable verilebilir ama limitsiz swap ya da başka stratejiler gerekir. En doğrusu: uygun --memory limiti ve uygulamada bellek profilli kod.

11.10 Logging Drivers

Driver	Kullanım
json-file	Varsayılan; /var/lib/docker/... altında JSON dosyası.
local	json-file'a göre daha verimli binary format.
syslog	Host syslog'una gönderir.
journald	systemd-journald'a yazar.
fluentd	Fluentd sunucusuna push.
gelf	Graylog / Logstash için GELF formatı.

Driver	Kullanım
awslogs, gcplogs, splunk	Doğrudan ilgili bulut servisine.

11.11 Log Rotation (Kritik!)

⚠ **Dikkat:** Varsayılan json-file driver rotation yapmaz. Çok loglayan bir konteyner host diskini doldurabilir. Mutlaka şunu ayarlayın:

```
# /etc/docker/daemon.json (daemon seviyesi, tüm konteynerler için)
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}

# veya tek konteyner için
docker run --log-driver=json-file \
  --log-opt max-size=10m --log-opt max-file=3 nginx
```

11.12 Monitoring

- docker stats: Anlık bakış.
- cAdvisor: Google tarafından geliştirilen konteyner metrikleri toplayıcı; Prometheus ile iyi çalışır.
- node_exporter + cAdvisor + Prometheus + Grafana: Açık kaynak standart yığını.
- Datadog, New Relic, Dynatrace: Ticari APM çözümleri.

11.13 Secret Yönetimi — En İyi Pratikler

- ENV yoluyla şifre geçirmeyin — docker inspect ile görünür.
- Dockerfile'a şifre yazmayın — docker history ile görünür.
- BuildKit secret mount, Docker secret (swarm/compose), veya external vault (HashiCorp Vault, AWS Secrets Manager) kullanın.
- Uygulamanızı POSTGRES_PASSWORD_FILE gibi _FILE sonekli değişkenlerle besleyin; böylece secret dosyadan okunur, ortam değişkeninde belirmez.

11.14 Lab: Güvenlik Sertleştirmesi Uygulama

```
# Önce normal (zayıf) bir nginx çalıştır
docker run -d --name soft -p 8081:80 nginx:1.27-alpine

# Şimdi sertleştirilmiş versiyonu
docker run -d --name hard -p 8082:80 \
  --read-only --tmpfs /var/cache/nginx --tmpfs /var/run --tmpfs /tmp \
  --cap-drop=ALL --cap-add=NET_BIND_SERVICE \
  --security-opt no-new-privileges:true \
  --memory=128m --cpus=0.5 --pids-limit=50 \
  --log-opt max-size=5m --log-opt max-file=2 \
  nginx:1.27-alpine

docker exec hard sh -c 'touch /x || echo READ ONLY olduğu için yazılamıyor'
```

Modül 12 — Registry, CI/CD ve Production Best Practices (15:15 – 16:30)

12.1 Kendi Private Registry'nizi Kurmak

```
# Docker'ın resmi registry:2 imajıyla
docker run -d --restart=always --name registry \
  -v regdata:/var/lib/registry \
  -p 5000:5000 registry:2

# Kullanma
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
docker pull localhost:5000/myapp:1.0

# HTTPS kurulumu için sertifikalarla detaylı konfigürasyon gerekir;
# production için Harbor veya Nexus gibi platformlar önerilir.
```

12.2 Harbor ve Kurumsal Registry'ler

- Harbor (CNCF): RBAC, replikasyon, zaafiyet tarama, image signing; açık kaynak.
- JFrog Artifactory: Generic binary repository; Docker dahil birçok format.
- Sonatype Nexus: Benzer, yaygın kurumsal çözüm.

12.3 Image Signing ve Trust

- Docker Content Trust (DCT) / Notary: Klasik ama son yıllarda yavaşladı.
- Sigstore / Cosign: Modern, keyless imzalama; GitHub OIDC ile entegre; CNCF projesi.

```
cosign sign ghcr.io/kullanici/myapp:1.0
cosign verify --certificate-identity="..." ghcr.io/kullanici/myapp:1.0
```

12.4 CI/CD'de Docker: GitHub Actions Örneği

```
# .github/workflows/docker.yml
name: build-and-push
on:
  push:
    branches: [main]
    tags: ["v*"]
jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/metadata-action@v5
        id: meta
        with:
```

```
        images: ghcr.io/${{ github.repository }}
    - uses: docker/build-push-action@v6
      with:
        context: .
        platforms: linux/amd64,linux/arm64
        push: true
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}
        cache-from: type=gha
        cache-to: type=gha,mode=max
```

12.5 GitLab CI Örneği

```
build:
  image: docker:27
  services:
    - docker:27-dind
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""
  before_script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --
password-stdin $CI_REGISTRY
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

12.6 Production Best Practices Kontrol Listesi

- latest tag'i kullanmayın — her deploy'da hangi imajın geldiği belirsiz olur. Semantic version veya commit SHA kullanın.
- İmajları mümkünse digest ile (sha256:...) referans edin.
- Multi-stage build + alpine/distroless → küçük imaj.
- Non-root USER, read-only FS, --cap-drop=ALL + gerekli cap'ler.
- Her konteynere --memory ve --cpus limiti.
- HEALTHCHECK tanımlı olsun; Compose'da depends_on ile condition: service_healthy.
- Log rotation yapılandırılmış; loglar stdout/stderr'e yazılıyor.
- Secrets env'de değil, secret mount'ta.
- Düzenli imaj taraması (Trivy, Scout) CI'da kırılma kriteri olarak çalışsın.
- Base image'ları düzenli olarak güncelleyin (weekly auto-rebuild).
- Dockerfile'ı çok katmanlı değil, okunabilir ve küçük tutun; her RUN'da cache temizliği yapın.
- .dockerignore ile build context'i yalın tutun.

12.7 Troubleshooting Akışı

Konteyner başlamıyor / hemen exit ediyor

14. docker ps -a ile exit code'a bakın (137=OOM/kill, 139=segfault).
15. docker logs <container> ile çıktıyı okuyun.
16. docker inspect <container> ile State.Error, Path, Args alanlarına bakın.
17. docker run --rm -it <image> sh ile aynı imajı manuel çalıştırıp içinde deneyin.

Ağ sorunu

18. docker network inspect <net> ile doğru ağda olup olmadığını kontrol edin.
19. docker exec <container> getent hosts <servisadi> ile DNS çözümlemesini test edin.
20. docker run --rm --network container:<target> nicolaka/netshoot ile hedefin ağ namespace'inde tcpdump/curl/nslookup yapın (distroless imajlarda kritik).

Volume / izin sorunu

21. docker exec -u 0 ... ile root olarak girip ls -la yapın.
22. Host UID ile konteyner UID'sinin uyuşup uyuşmadığına bakın.
23. SELinux'lu sistemlerde -v HOST:CONT:Z kullanılmış mı kontrol edin.

12.8 daemon.json ile Docker Daemon Yapılandırması

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "storage-driver": "overlay2",
  "live-restore": true,
  "registry-mirrors": ["https://mirror.gcr.io"],
  "insecure-registries": ["registry.local:5000"],
  "default-ulimits": { "nofile": { "Name": "nofile", "Soft": 65536, "Hard": 65536 } },
  "dns": ["1.1.1.1", "8.8.8.8"],
  "userns-remap": "default"
}

# Değiştirdikten sonra:
sudo systemctl restart docker
```

Not: live-restore: true, dockerd yeniden başlatıldığında (güncelleme vs.) konteynerlerin çalışmaya devam etmesini sağlar — üretimde hayat kurtaran ayar.

12.9 Bir Sonraki Adım: Kubernetes'e Köprü

Compose ile öğrendiğiniz "servis, ağ, volume, healthcheck, resource limit" kavramları birebir Kubernetes'te Pod, Service, PersistentVolume, livenessProbe/readinessProbe ve resources.limits/requests olarak karşınıza çıkar. Docker'da ne kadar sağlam temel atarsanız Kubernetes o kadar kolaylaşır. kompose adlı araç, compose.yaml'ı Kubernetes manifest'lerine çeviren pratik bir başlangıç noktasıdır.

12.10 Kapanış ve Soru-Cevap

Üç günde bir konteyneri çalıştırmaktan, güvenli bir şekilde build edip, birden fazla servisle orkestra edip, production standartlarıyla izlemeye kadar uzandık. Kalıcılık için önerim: Önümüzdeki iki hafta her gün en az 15 dakika Docker ile kendi küçük bir projenizi konteynerleştirin. Kaslar alışkanlıkla şekillenir.

Ek — Kısa Komut Referansı

A.1 En Sık Kullanılan 30 Komut

```
# Kurulum ve bilgi
docker --version
docker info

# İmaj
docker pull <image>
docker images
docker rmi <image>
docker image prune -a
docker tag <src> <dst>
docker history <image>
docker inspect <image>
docker save -o out.tar <image>
docker load -i out.tar

# Build
docker build -t <tag> .
docker buildx build --platform linux/amd64,linux/arm64 -t <tag> --push .

# Konteyner
docker run -d --name <n> -p 8080:80 --restart unless-stopped <image>
docker ps / docker ps -a
docker stop / start / restart / kill <n>
docker rm -f <n>
docker exec -it <n> sh
docker logs -f --tail 100 <n>
docker inspect <n>
docker stats
docker cp <src> <dst>

# Volume / Network
docker volume create / ls / inspect / rm / prune
docker network create / ls / inspect / connect / disconnect / rm / prune

# Compose
docker compose up -d
docker compose down -v
docker compose ps / logs -f / exec <svc> sh
docker compose build / pull / config

# Sistem bakımı
docker system df
docker system prune -a --volumes
```

A.2 Önerilen Kaynaklar

- docs.docker.com – resmi dokümantasyon.
- github.com/docker-library – resmi imajların Dockerfile'ları (altın kaynak).
- Docker Deep Dive — Nigel Poulton (kitap).

- play-with-docker.com – tarayıcıdan ücretsiz Docker lab.
- hadolint — Dockerfile lint aracı: github.com/hadolint/hadolint.
- dive — imaj katmanlarını görselleştiren CLI aracı: github.com/wagoodman/dive.
- nicolaka/netshoot — konteyner ağı debug için çok yönlü imaj.

A.3 Değerlendirme Sertifikası İçin Mini Sınav Soruları

24. Bir container ile bir VM arasındaki üç temel farkı açıklayın.
25. CMD ve ENTRYPOINT arasındaki farkı örnekle anlatın.
26. docker image prune ile docker image prune -a arasındaki fark nedir?
27. Default bridge ile user-defined bridge arasındaki iki büyük fark nelerdir?
28. Neden Dockerfile'da hassas veriyi ARG/ENV ile değil secret mount ile geçiririz?
29. Multi-stage build'in amacı nedir, basit bir örnekle anlatın.
30. docker logs -f ile docker stats arasındaki fark nedir?
31. depends_on tek başına neden yeterli değildir, condition: service_healthy'yi ne zaman kullanırız?
32. --read-only ile --cap-drop=ALL kombinasyonu hangi saldırı vektörlerini neden azaltır?
33. latest tag'i production'da neden kötü bir fikirdir, yerine ne kullanılmalıdır?